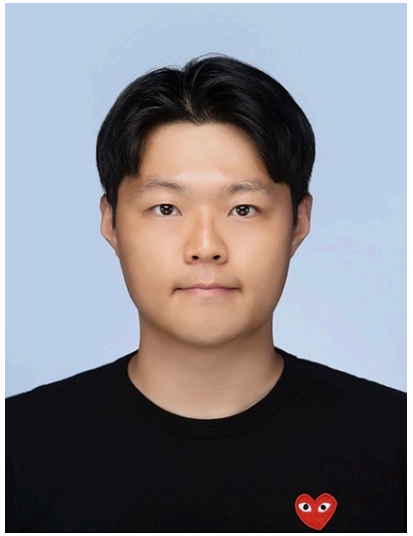
Effective and Efficient Out-of-Core Training for Large-Scale Graph Neural Networks

2026.10.28

Saejoon Park
Major in Big Data
Department of Computer Science and Engineering
Chung-Ang University

Information



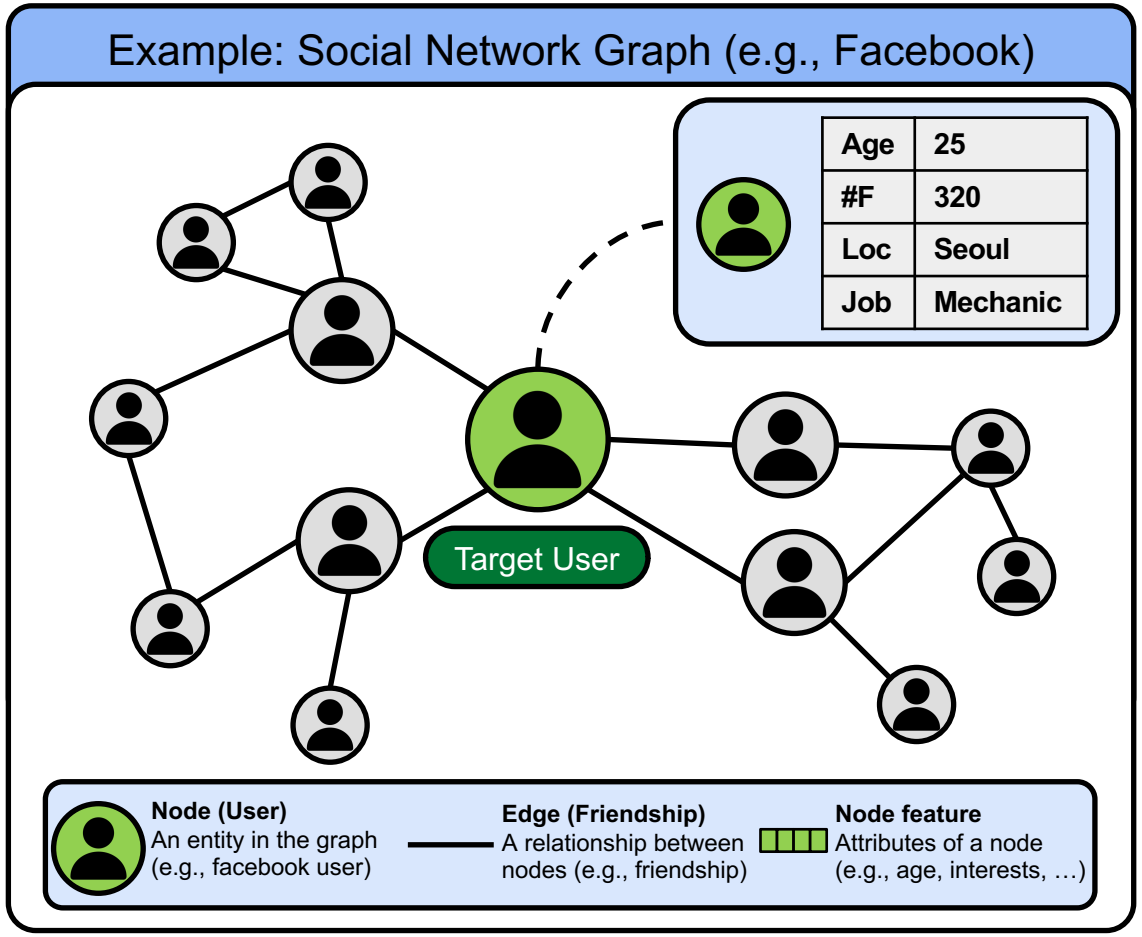
- **Name: Park SaeJoon**
- **Birth: 1999.07.14**
- **Education:**
  - 2019.03 ~ 2025.02 Chung-ang Univ Undergraduate program
  - 2025.03 ~ Present Chung-ang Univ Master's Student
- **Major: Big Data**
- **Research Area: GNN Acceleration, Time Series Anomaly Detection**
- **Papers:**
  - S. Park, S. Lee, G. Lee, and Y. Ko, "FALCON: Fast and scalable out-of-core training of graph neural networks," submitted for publication in IEEE Access.
  - G. Lee, S. Park, W. Hwang, B. Kim and Y. Ko, "TriAD: Tri-perspective learning for multivariate time series anomaly detection," manuscript in preparation, intended for submission to Knowledge-Based Systems.
  - S. Park, G. Lee, and Y. Ko, "An empirical analysis of score mapping in time-series anomaly detection," in Proc. Int. Conf. Inf. Commun. Technol. Converg. (ICTC), 2025, pp. 907–910.

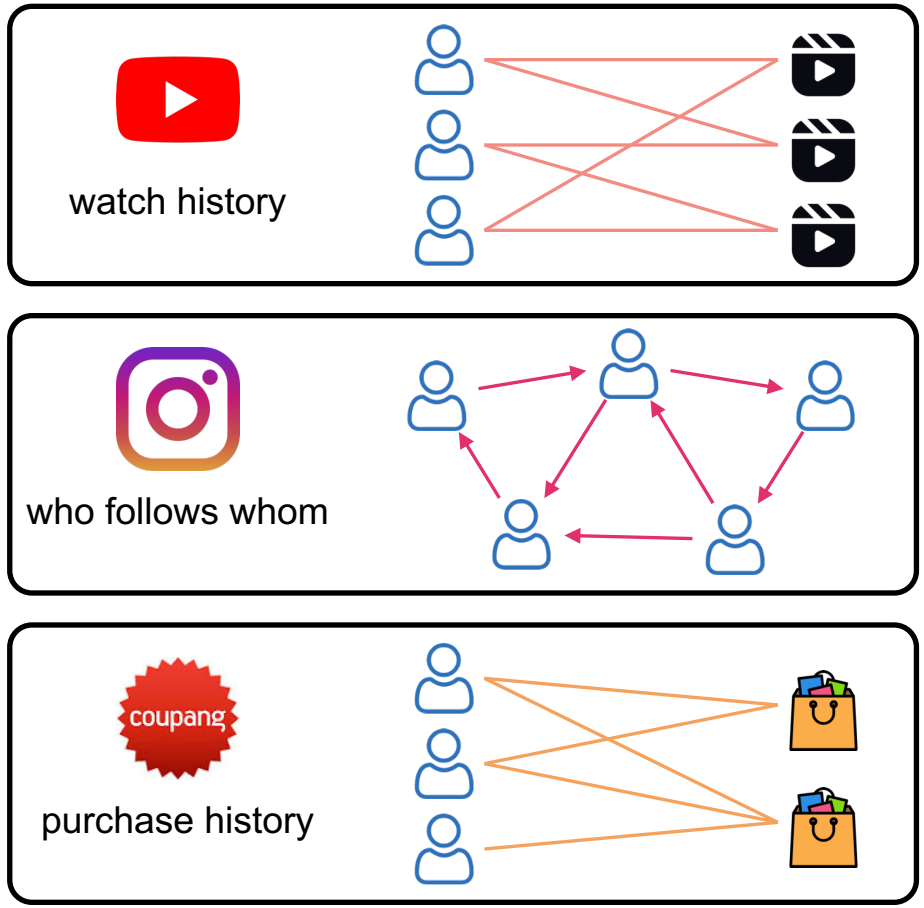
Contents

- **Graph Data and Graph Neural Network**
  - Graph data
  - Graph Neural network
- **In Memory Mini-Batch Training and Out of Core Training**
  - Sampling based graph learning
  - Mini batch training and problem
  - Out of core training
- **FALCON**
  - Remaining Challenges
  - All about Falcon
- **Experiments**
  - Eq 1
  - Eq 2
- **Conclusion**

What is Graph Data?

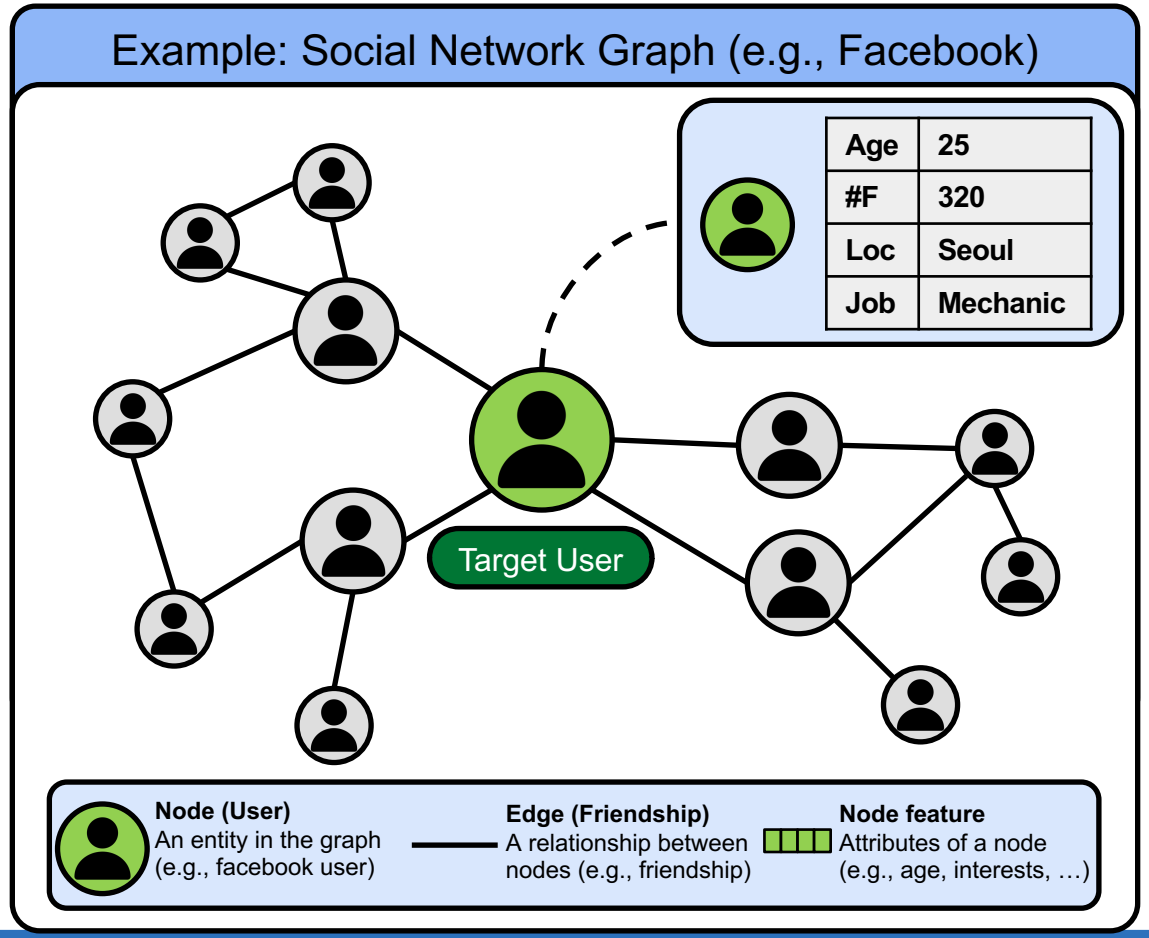
- Graph data represents entities as nodes and relationships between them as edges
- Each node can have features describing its characteristics, while edges capture the connections between nodes

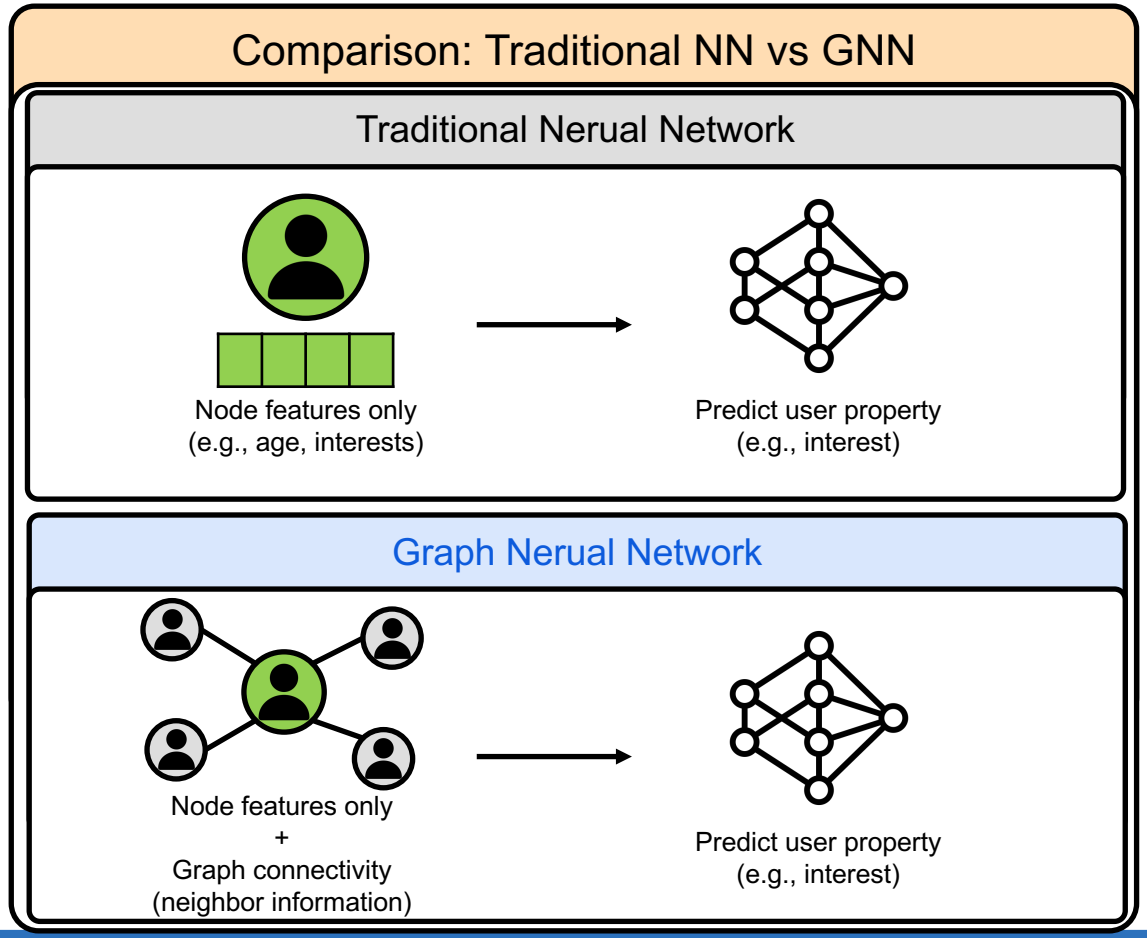




What is a Graph Neural Network?

- **Graph Neural Networks (GNNs)** are neural networks designed to learn from graph-structured data
- Unlike traditional neural networks, GNNs leverage both node features and graph connectivity
- GNNs learn node representations for tasks such as node classification, link prediction, and recommendation

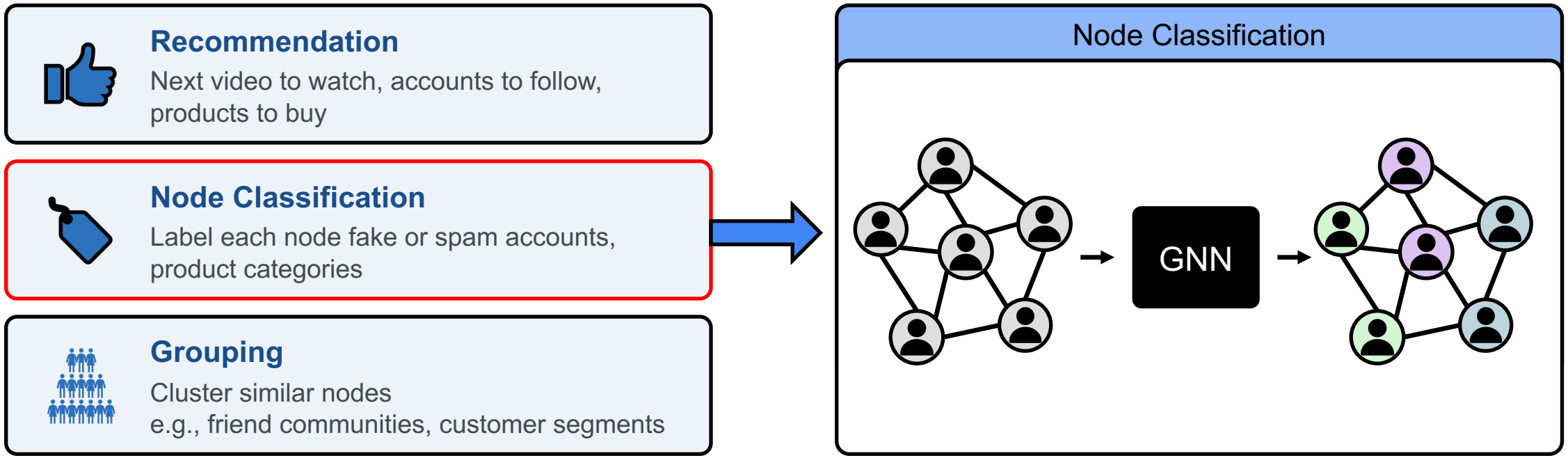




Reference: "Z. Wu et al., "A Comprehensive Survey on Graph Neural Networks," IEEE TNNLS, 2021.";
"F. Scarselli et al., "The Graph Neural Network Model," IEEE TNN, 2009."

Applications of GNNs

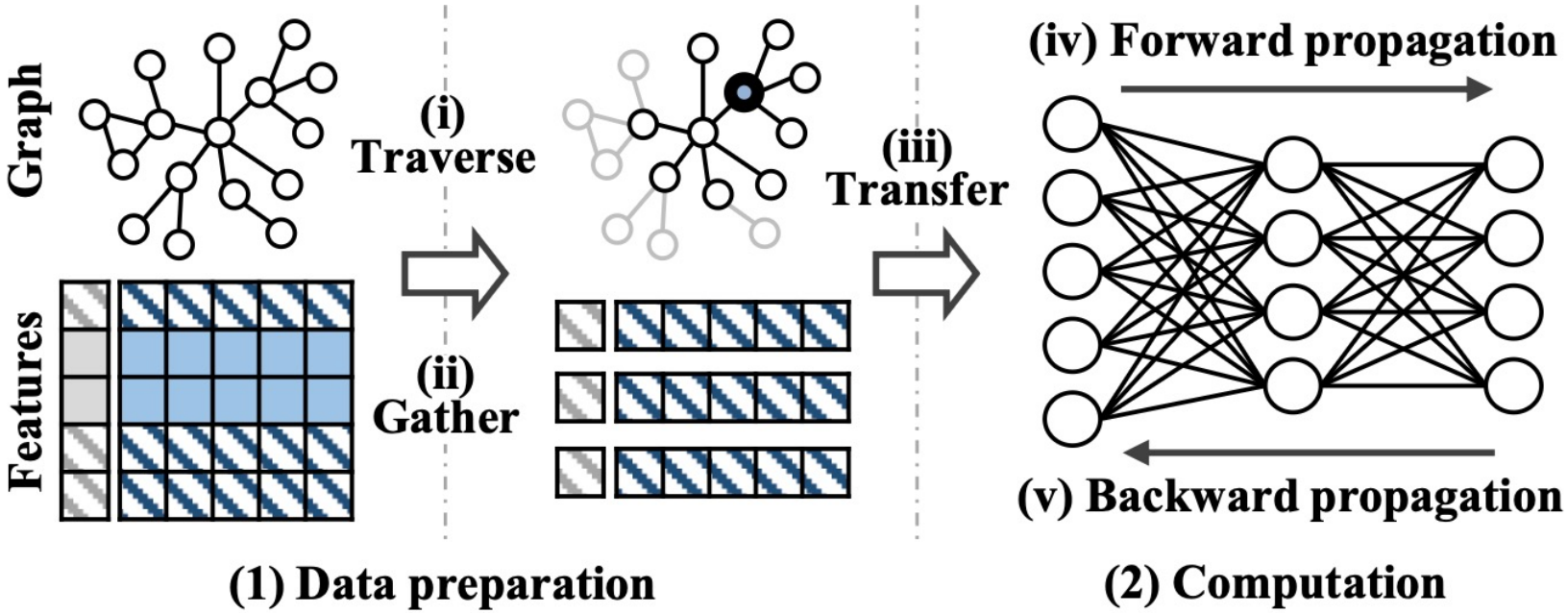
- Recommendation: Predicts user preferences based on interactions between users and items
- Grouping: Groups nodes with similar characteristics or relationships
- Node Classification: Predicts the class of each node using its features and graph connections
- Node Classification Example: Classifies social network users based on their interests and connections with other users



Reference: “T. N. Kipf and M. Welling, ″GCN,″ ICLR, 2017”; “W. Hamilton et al., ″GraphSAGE,″ NeurIPS, 2017”; “Y. Zhang et al., ″SEAL,″ KDD, 2020.”

How Does GNN Message Passing Work?

- GNN training consists of two main stages: data preparation and model computation
- Data preparation involves graph traversal, neighbor sampling, and feature gathering
- Model computation performs message passing, aggregation, and node representation updates
  - Forward and backward propagation



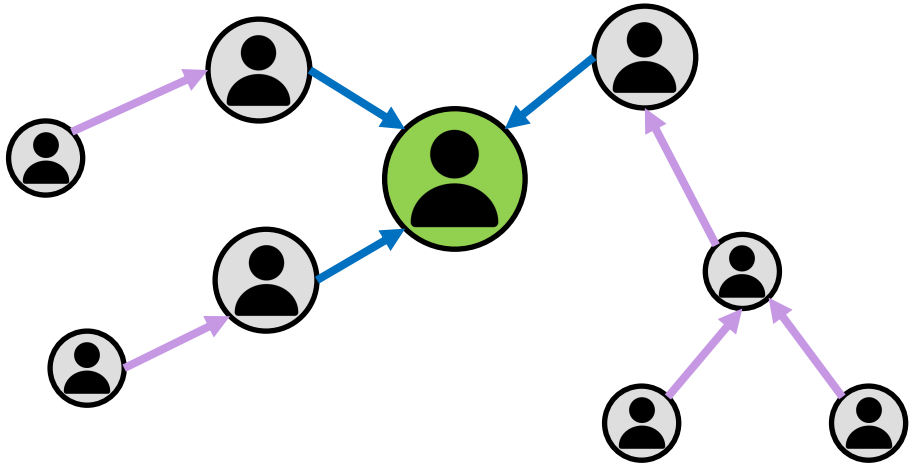
1. Graph Traversal
2. Feature Gathering
3. Message Computation
4. Scatter / Aggregation
5. Node Representation Update

Reference: “M.-H. Jang et al., ”Accelerating Storage-Based Training for Graph Neural Networks,” KDD, 2026.”

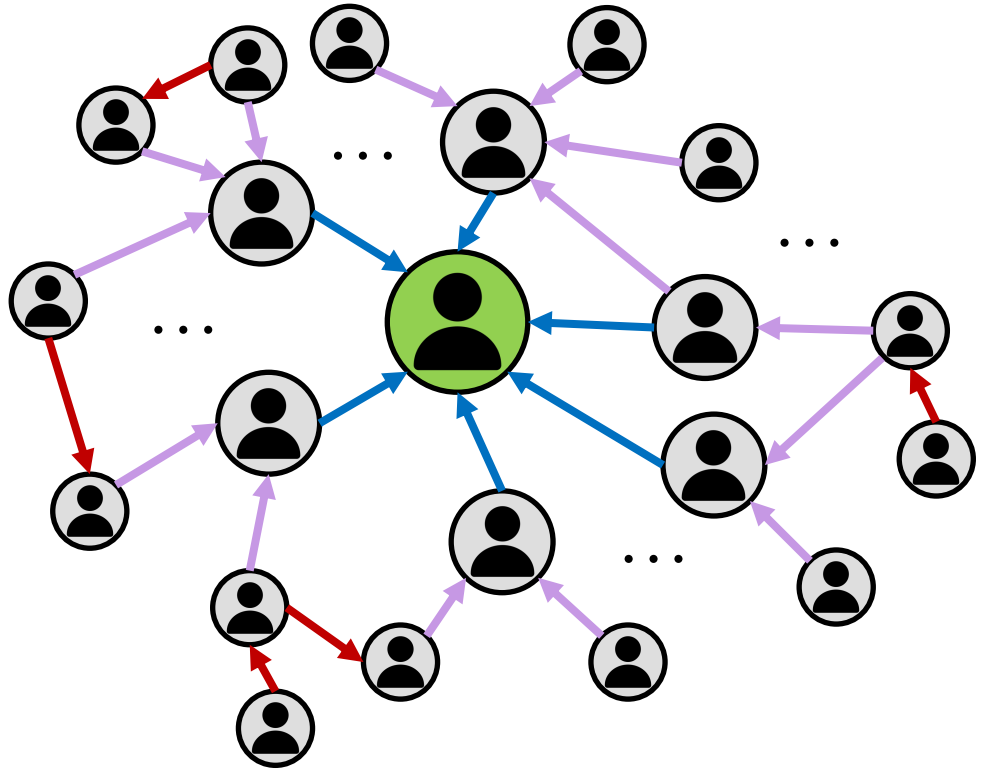
Message Passing in Small and Large-Scale Graphs

- Small graphs require relatively few neighbor accesses for message passing
- Large-scale graphs increase neighbor traversal and feature access costs
- Memory, I/O, and computation overhead become major training bottlenecks

Small Graph Message passing

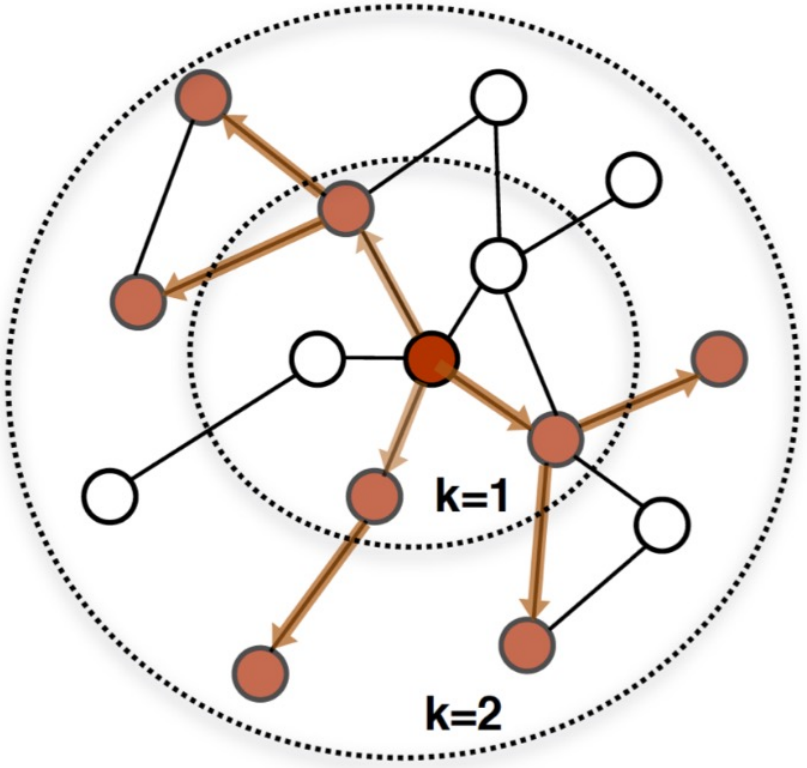


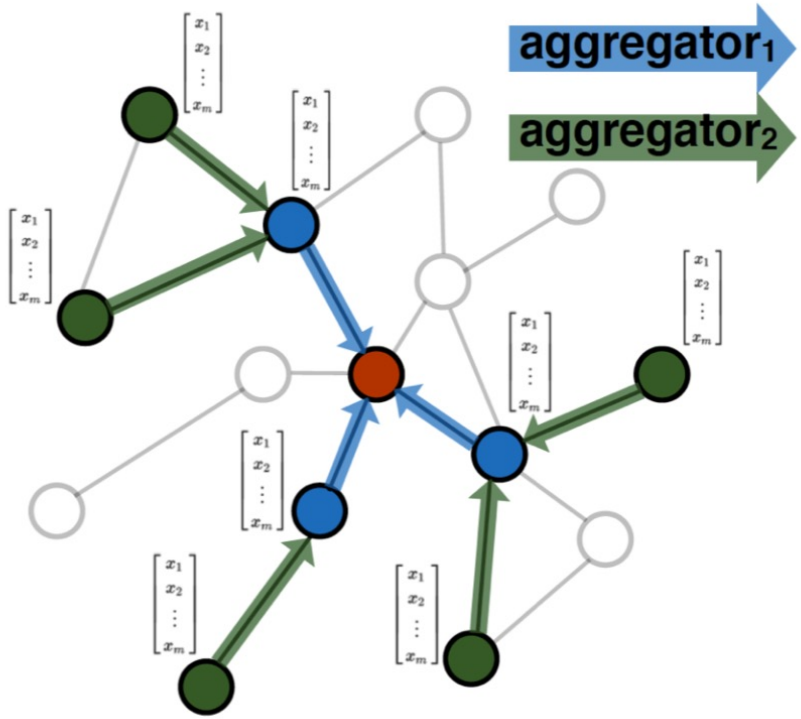
Large scale Graph Message passing



Sampling Based Graph Learning

- Sample up to a fixed number of neighbors at each hop
- Aggregate information from sampled neighbors to compute node representations
- Use the learned representations for prediction and model training
- Frameworks such as PyG and DGL support GNN training with neighbor sampling

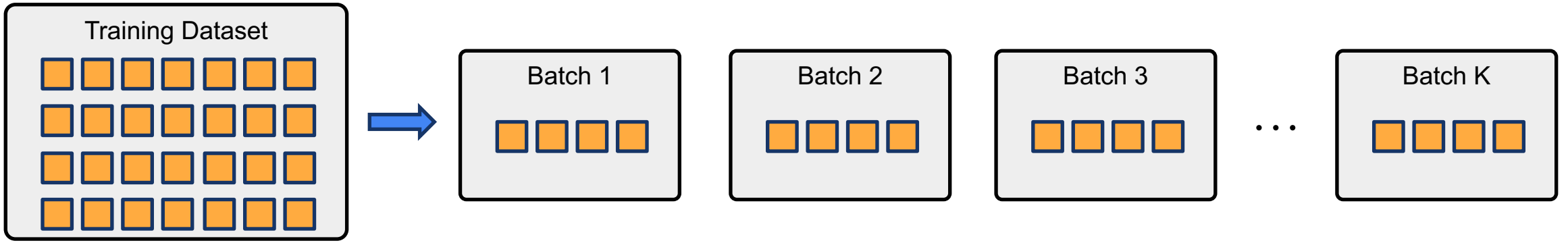




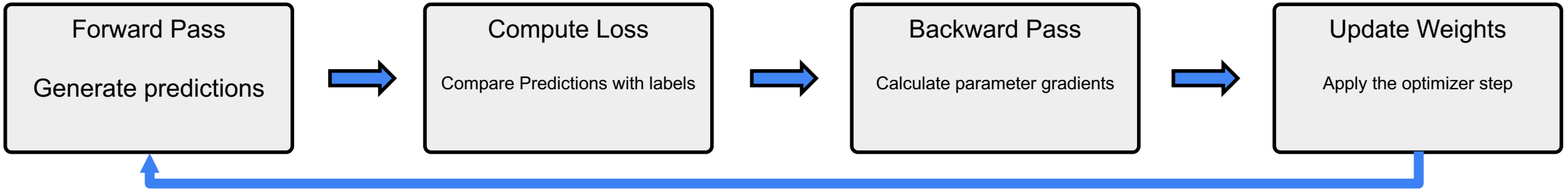
Mini Batch Training

- Divide training nodes into mini-batches to enable efficient GNN training
- Each mini-batch updates model parameters through forward and backward propagation
- One epoch is completed when all training mini-batches have been processed once

1. Prepare the batches



2. Train on one mini-batch

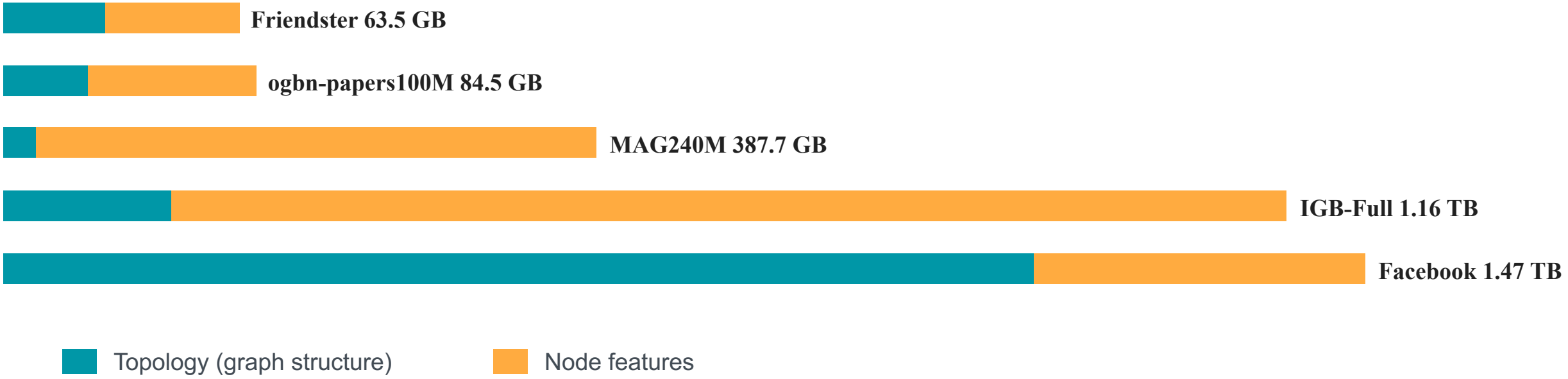


Next mini batch, using the update weights

What about Large Scale Graph Data?

- Datasets nowadays are far larger than the memory of an ordinary machine

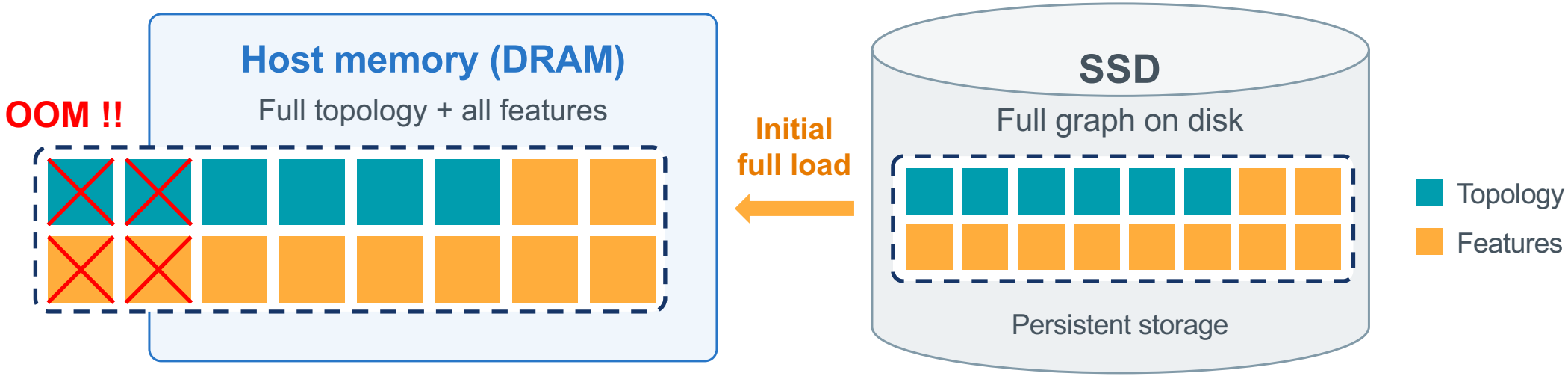
Dataset	Nodes	Edges	Feature Dim.	Topology	Features	Total Memory
*Friendster	65.6M	1.81B	128	29.9 GB	33.6 GB	63.5 GB
ogbn-papers100M	111.1M	1.62B	128	27.6 GB	56.9 GB	84.5 GB
MAG240M	244.2M	1.73B	768	15.8 GB	371.9 GB	387.7 GB
IGB-Full	269.3M	4.00B	1024	56 GB	1.1 TB	1.16 TB
*Facebook (Estimated)	721.1M	137.33B	128*	1.1 TB	369 GB	1.47 TB



Reference: “J. Yang and J. Leskovec, ”Network Communities,” ICDM, 2012”; “W. Hu et al., ”Open Graph Benchmark,” NeurIPS, 2020.”

In Memory Mini-batch Training's Problem

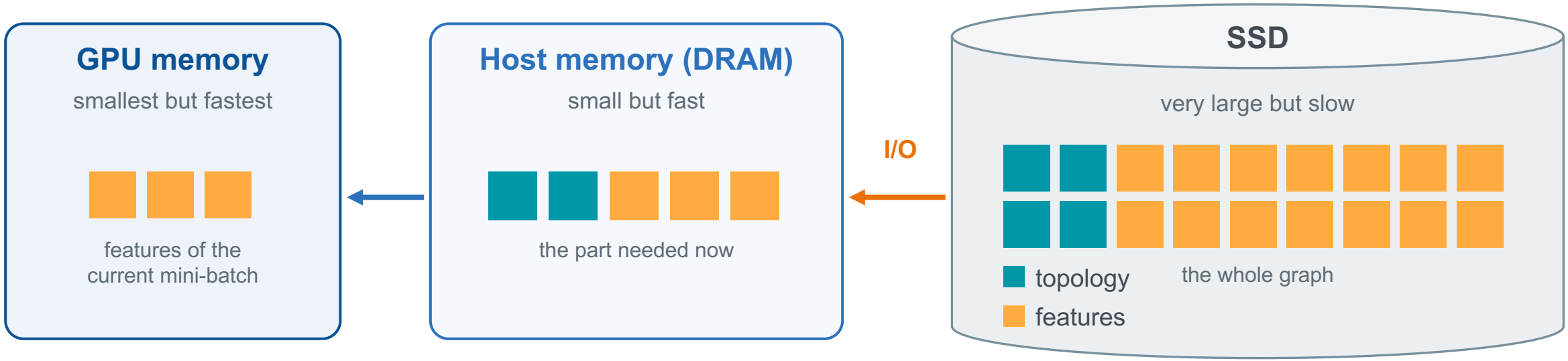
- The full graph structure and features must fit in host memory
- If sampling and data transfer cannot keep up with GPU computation, the GPU waits for the next mini-batch
- PyG: Typically loads graph topology and node features into host memory for mini-batch training
- DGL: Also relies on in-memory graph data, limiting scalability when graphs exceed host memory



Reference: "Y. Park et al., "Ginex," PVLDB, 2022."; "R. Waleffe et al., "MariusGNN," EuroSys, 2023."

Out-of-Core Training

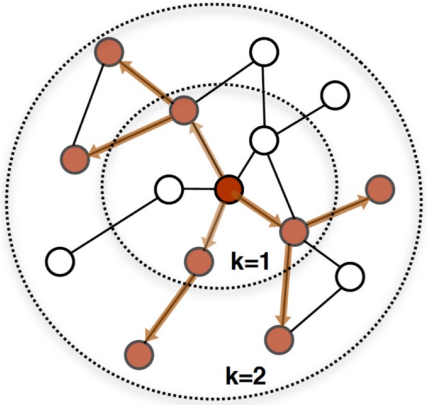
- Out-of-core training stores large-scale graph data on SSD and loads only the data required for each mini-batch
- Frequent SSD accesses introduce significant I/O overhead during GNN training
- Ginex: Uses SSD-based graph storage and feature caching to reduce redundant I/O
- MariusGNN: Uses graph partitioning and data ordering to minimize disk I/O during training

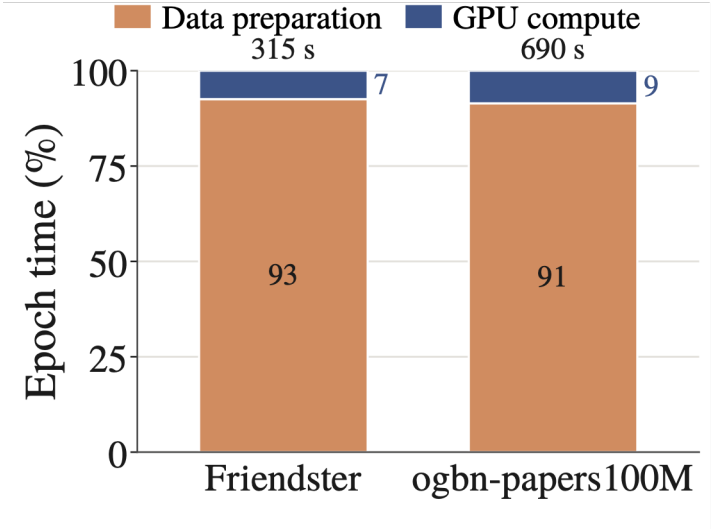


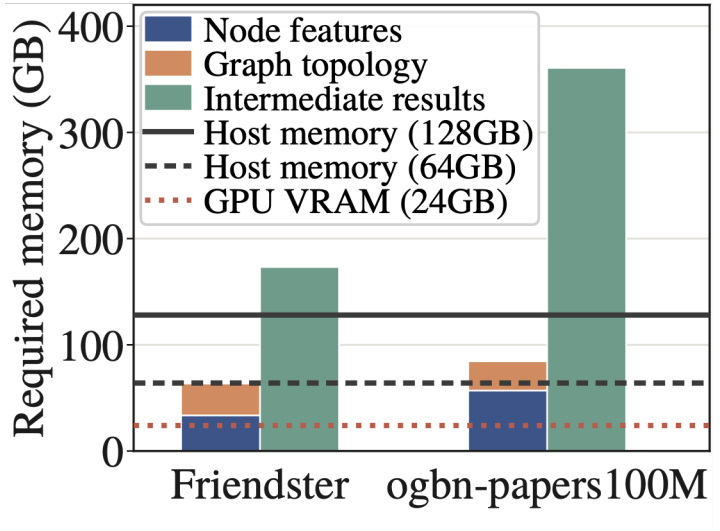
Reference: “Z. Sheng et al., ″OUTRE,″ PVLDB, 2024.”, “Y. Park et al., ″Ginex,″ PVLDB, 2022.”, “R. Waleffe et al., ″MariusGNN,″ EuroSys, 2023.”

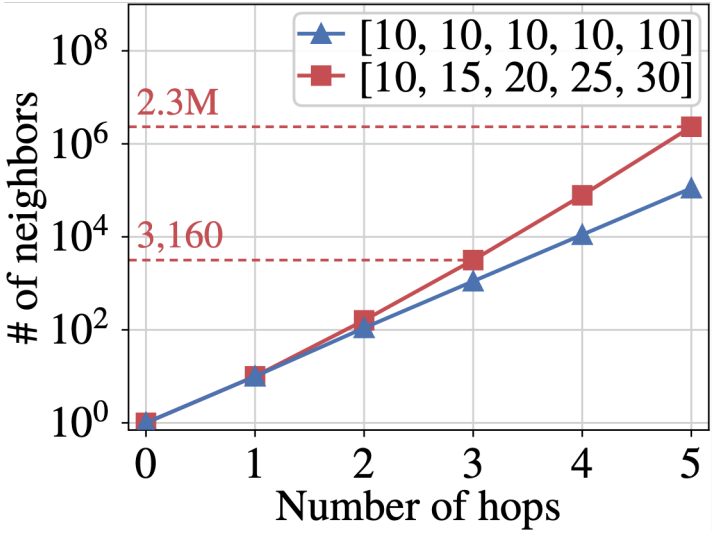
Remain Two Major Challenges

- **C1 Large Underlying Graph Size**
  - Message passing requires repeated access to graph topology and node features
  - Large-scale graphs incur substantial memory and I/O overhead during message passing
- **C2 Large Search Space for Neighbors**
  - High-degree nodes expand the neighbor search space, while large fanouts increase multi-hop sampling costs
  - Restricting the sampling space reduces overhead but may degrade model accuracy



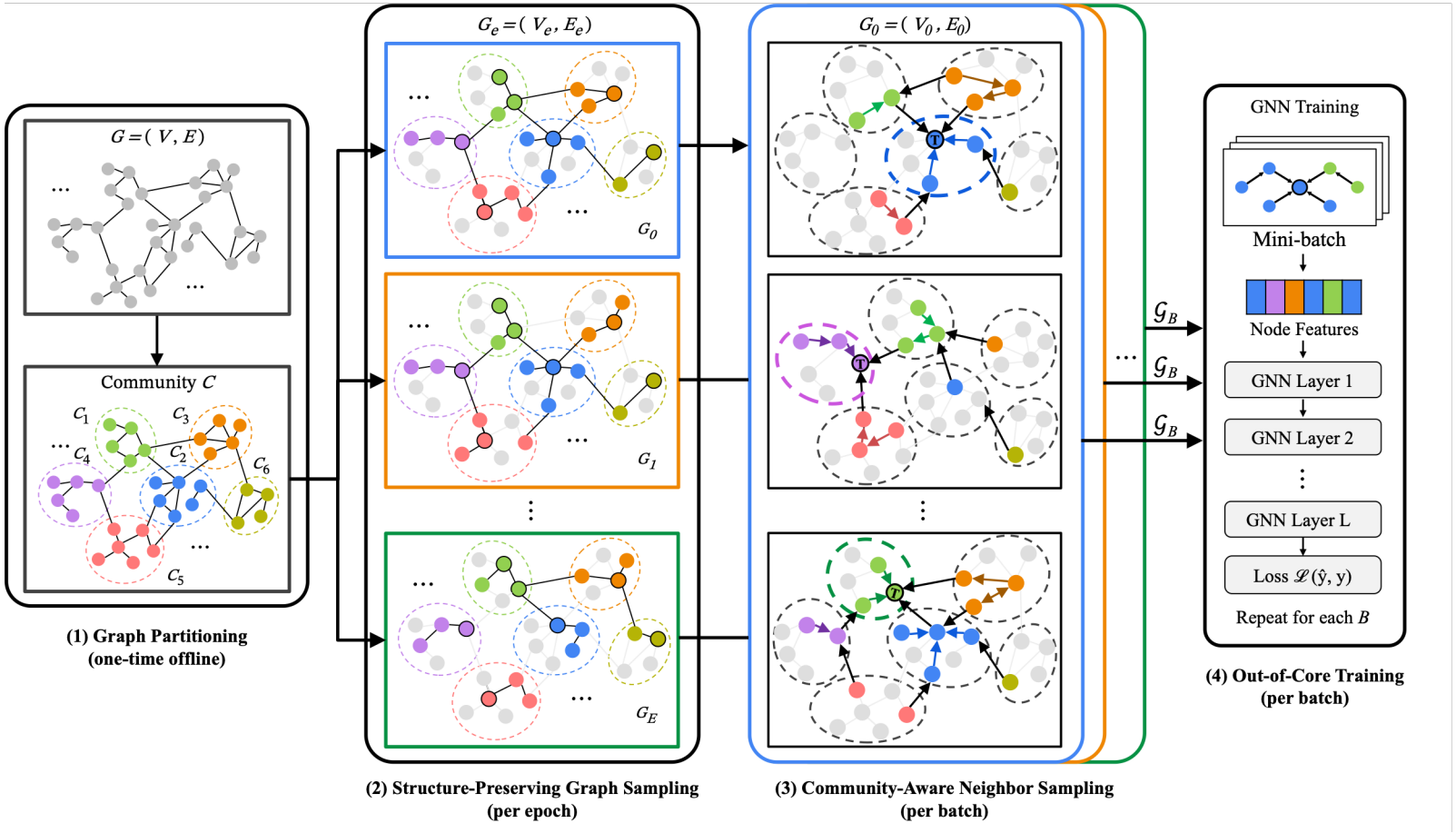






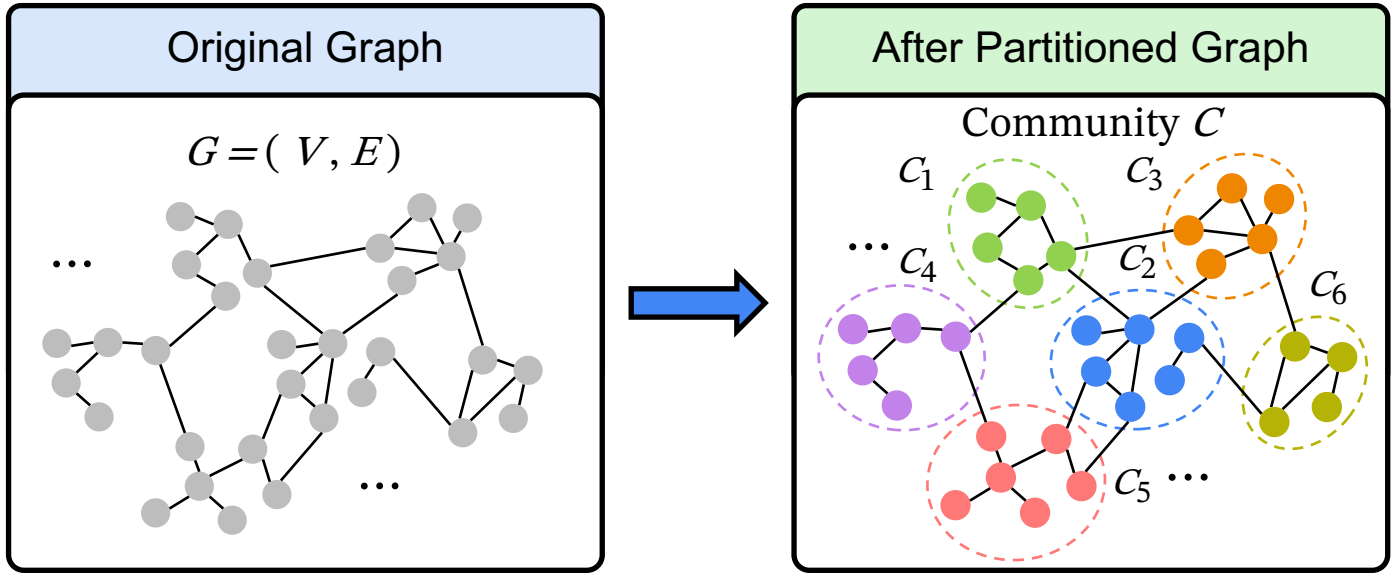
Overview

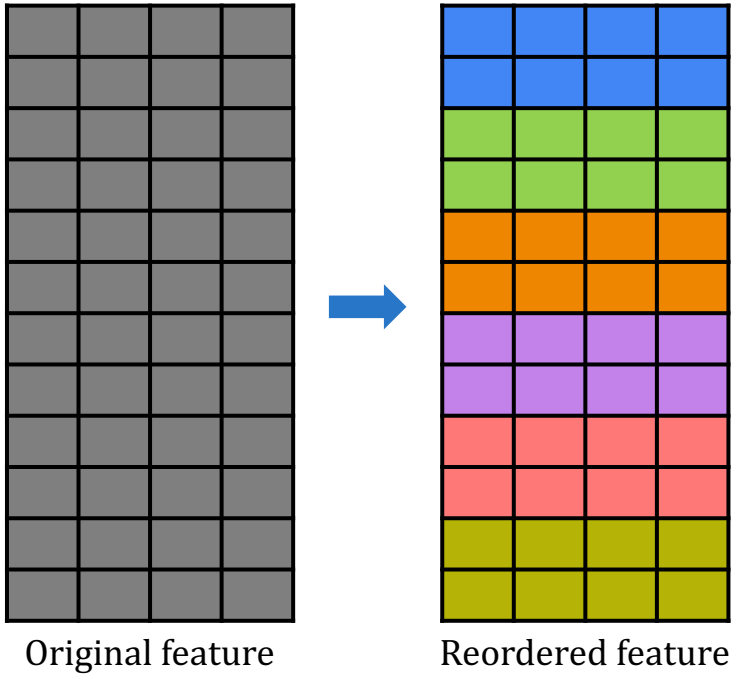
- FALCON is a storage-based framework for efficient large-scale GNN training
- Reduces graph size and sampling overhead through structure-preserving and community-aware sampling
- FALCON integrates these strategies with out-of-core training to accelerate GNN training



One Time Offline Clustering

- Nodes within the same community are more likely to provide relevant information for target nodes
- Identifies communities for subsequent structure-preserving and community-aware sampling
- Reorders graph data based on communities to improve data locality and reduce random accesses during training
- One Time Offline Graph Partitioing with METIS
- Original node features are reordered according to the graph partitioning results

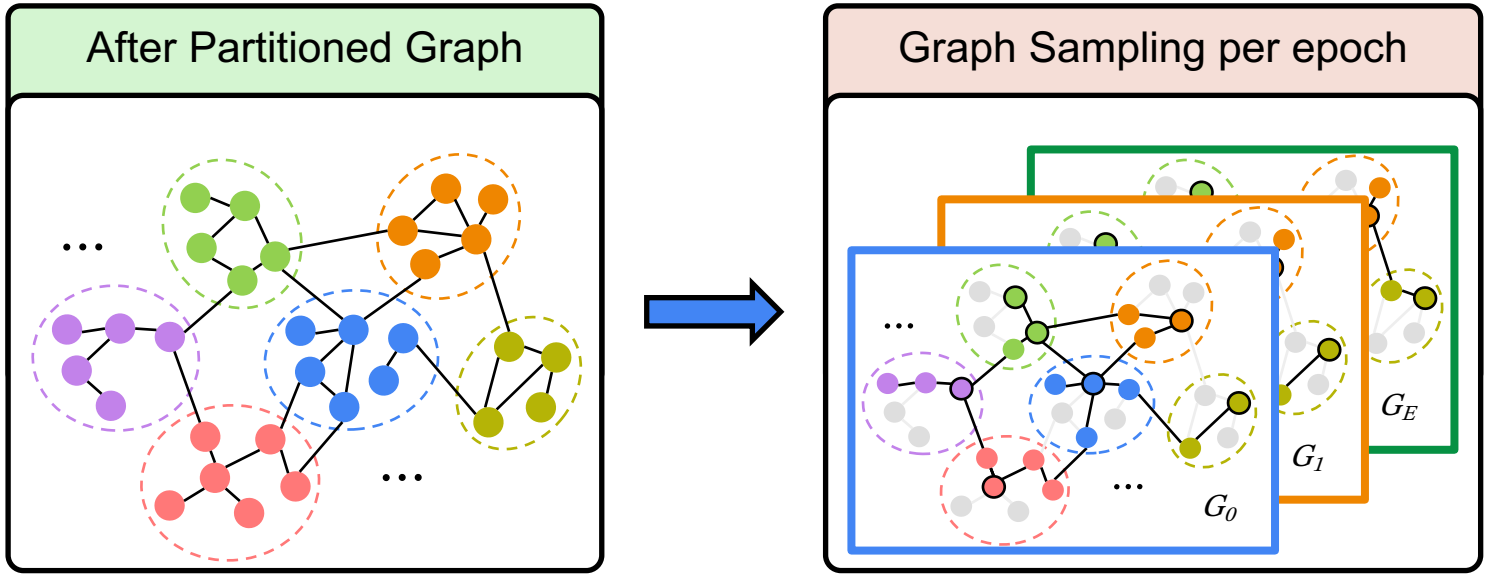




Reference: "W.-L. Chiang et al., "Cluster-GCN," KDD, 2019."

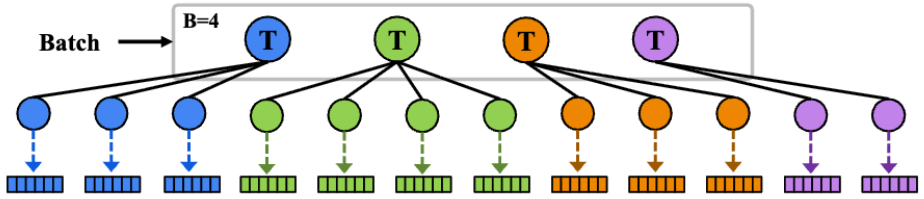
Structure Preserving Graph Sampling

- Per-epoch graph sampling increases sampling diversity by exposing the model to different graph structures
- If graph sampling is performed only once, limited training data diversity may degrade model accuracy
- Sample within each community to preserve community structure and retain all training nodes
- Resample the graph at the beginning of each epoch

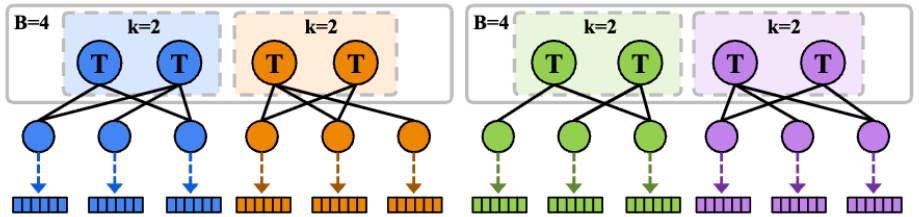


Make Blocks and Shuffle

- Low neighbor sharing leads to frequent random accesses and high I/O overhead
- Excessive neighbor sharing reduces sampling diversity and may degrade model accuracy
- Balancing neighbor sharing and diversity is essential for reducing I/O overhead while mitigating accuracy degradation



(a) Sampling without community-aware node blocking

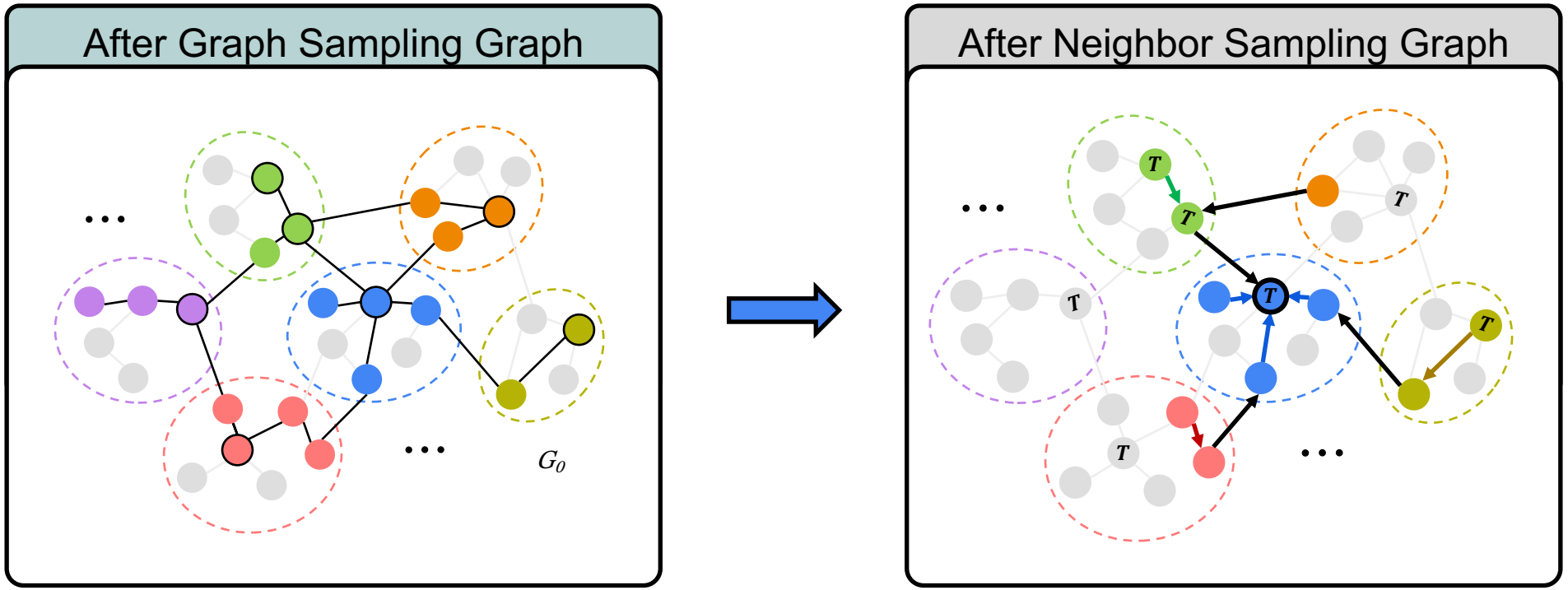


(b) Sampling with community-aware node blocking

Method	Unique Nodes
Global shuffle	$341{,}751 \pm 132$
Block only	$104{,}057 \pm 28$
FALCON	$231{,}046 \pm 191$

Community Aware Neighbor Sampling

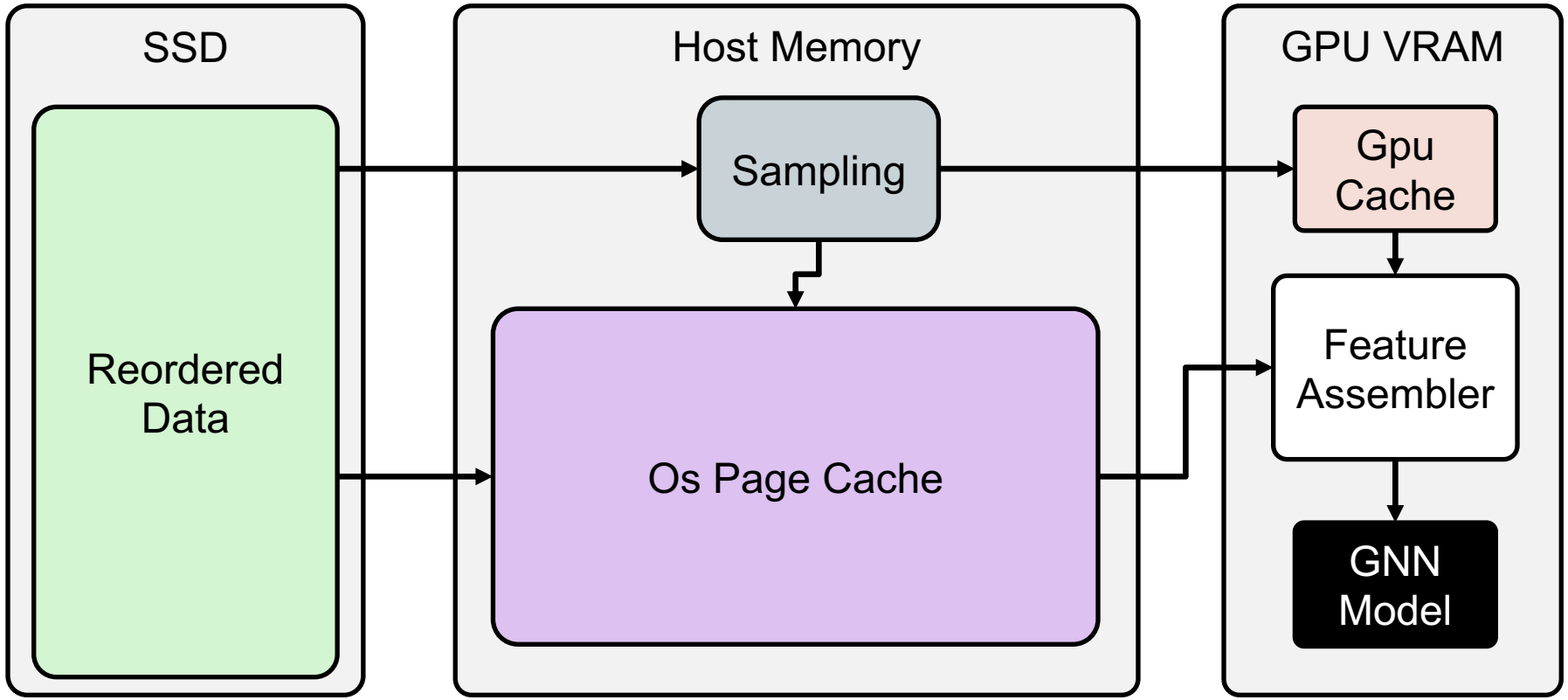
- Neighbor nodes may belong to different communities, resulting in a large sampling search space.
- Uniform random sampling may select neighbors across many communities, increasing random I/O overhead.
- Nodes within the same community are assumed to provide more relevant information for target nodes.
- Leverages community information to reduce random I/O while preserving relevant neighbors



Reference: “W.-L. Chiang et al., “Cluster-GCN,” KDD, 2019.”

Out of Core Training

- Loads graph data from SSD and performs graph sampling, block-and-shuffle, and neighbor sampling.
- Retrieves required features from two caches, assembles them on the GPU, and trains the GNN model.
- Uses LRU-based page eviction to enable training even when graph data exceeds host memory capacity.



Experiments

- **Our environments**
  - GPU: NVIDIA RTX 5080 (16 GB VRAM)
  - CPU: AMD Ryzen 9 9900X (12 Physical Cores)
  - Host Memory: 128 GB
  - Storage: PCIe Gen 4.0 NVMe SSD 2 TB

- **Used datasets**

Dataset	$\|V\|$	$\|E\|$	Topology	Feat.	Avg. Deg.
ogbn-products	2.45M	61.8M	1.0G	1.0G	50.5
ogbn-papers100M	111M	1.6B	27.6G	56.9G	29.1
Friendster	65.6M	1.8B	29.9G	33.6G	55.1

Dataset	$\|V_{\text{train}}\|$	$\|V_{\text{valid}}\|$	$\|V_{\text{test}}\|$
ogbn-products	197K (8.03%)	39.3K (1.61%)	2.21M (90.37%)
ogbn-papers100M	1.21M (1.09%)	125K (0.11%)	214K (0.19%)
Friendster	656K (1.00%)	–	–

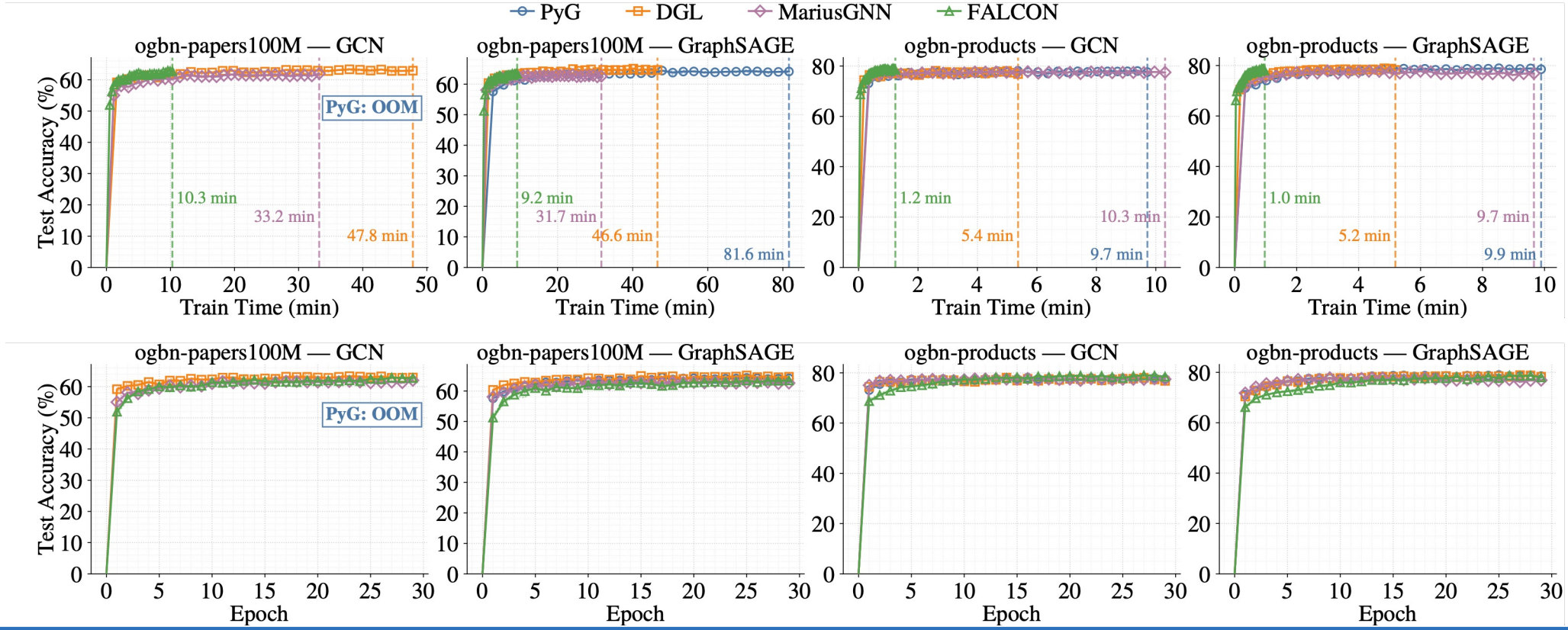
Experiments

- **EQ 1 Overall training efficiency**
  - FALCON achieves the shortest training time in most settings
  - PyG and DGL encounter OOM on large graphs with limited host memory
  - FALCON achieves up to 149.2× speedup over Ginex and 11.0× over MariusGNN, with Friendster at 64 GB as the only exception

Host Mem.	Model	Method	ogbn-products		ogbn-papers100M		Friendster	
			Time	Memory	Time	Memory	Time	Memory
64GB	GCN	PyG	20.79	11.9	–	OOM	–	OOM
		DGL	12.17	4.5	–	OOM	–	OOM
		Ginex	257.21	3.1	1139.89	21.1	673.74	10.0
		MariusGNN	22.44	3.3	75.21	12.3	**28.30**	7.0
		FALCON	**2.61**	2.9	**53.98**	12.1	33.04	9.3
	GraphSAGE	PyG	21.12	11.3	–	OOM	–	OOM
		DGL	11.90	5.5	–	OOM	–	OOM
		Ginex	260.13	3.0	1153.31	18.7	674.06	10.9
		MariusGNN	21.06	3.6	71.93	11.9	**27.42**	7.1
		FALCON	**2.63**	2.7	**54.54**	13.1	31.19	10.0
128GB	GCN	PyG	20.10	12.1	–	OOM	82.93	98.7
		DGL	11.10	4.8	99.00	83.2	52.34	63.7
		Ginex	247.62	3.1	1064.77	21.7	614.83	11.7
		MariusGNN	21.32	3.3	68.69	12.6	25.44	7.1
		FALCON	**2.32**	2.9	**20.02**	12.4	**15.35**	9.3
	GraphSAGE	PyG	20.46	10.9	168.89	113.4	85.98	99.6
		DGL	10.73	4.7	96.48	83.1	51.28	65.3
		Ginex	271.62	5.0	1149.20	21.0	608.07	9.9
		MariusGNN	19.98	3.3	65.44	11.6	24.21	7.7
		FALCON	**1.82**	2.8	**18.14**	12.3	**15.15**	8.8

Experiments

- **EQ 2 Convergence rate analysis**
  - FALCON achieves accuracy competitive with PyG and DGL while consistently outperforming MariusGNN in converged accuracy
  - On ogbn-papers100M, FALCON completes 30 epochs in 10.3 minutes for GCN and 9.2 minutes for GraphSAGE
  - Preserve structural information while reducing graph and neighbor search space for faster convergence with minimal accuracy loss



Conclusions

- **Research Achievements**
  - Began research in Time-Series Anomaly Detection (TSAD)
  - Strengthened research skills through participation in laboratory research projects
  - Focused on GNN acceleration, leading to research achievements and a paper submission

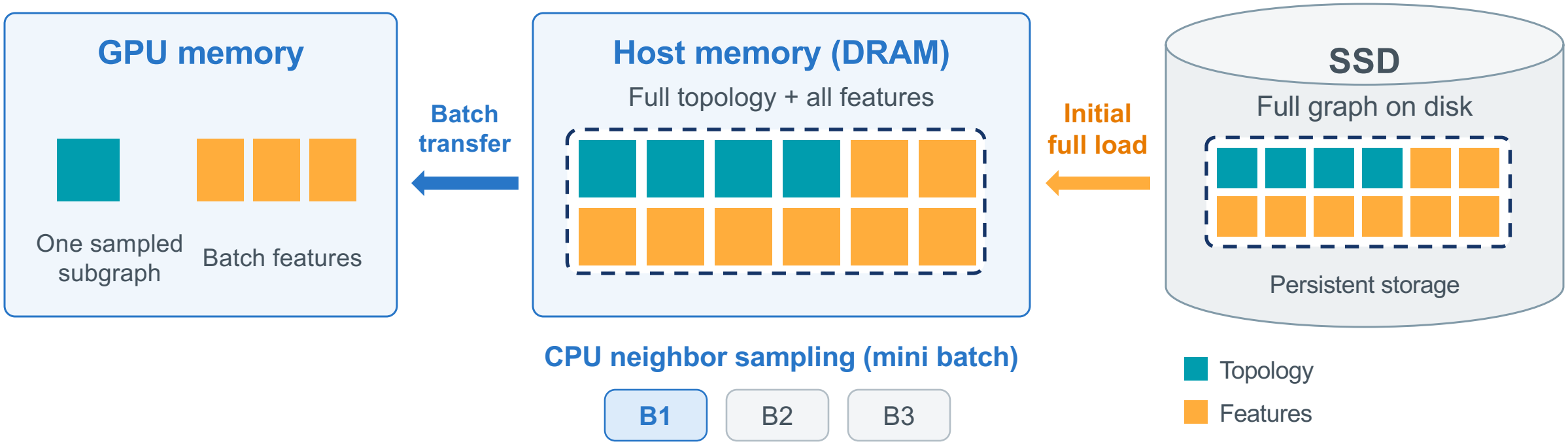
- **Lessons Learned**
  - Gained broad knowledge of graph structures, GNNs, and large-scale graph learning
  - Developed a deeper understanding of GNN training bottlenecks and effective acceleration strategies
  - Strengthened research, problem-solving, and experimental skills through hands-on research experience

- **Future Directions**
  - Explore further improvements to FALCON and GNN training efficiency during the remaining master's program
  - Pursue a career where I can apply and advance my research expertise in real-world AI systems
  - Achieve personal and professional fulfillment through continuous learning, innovation, and meaningful contributions

Appendix

PyG

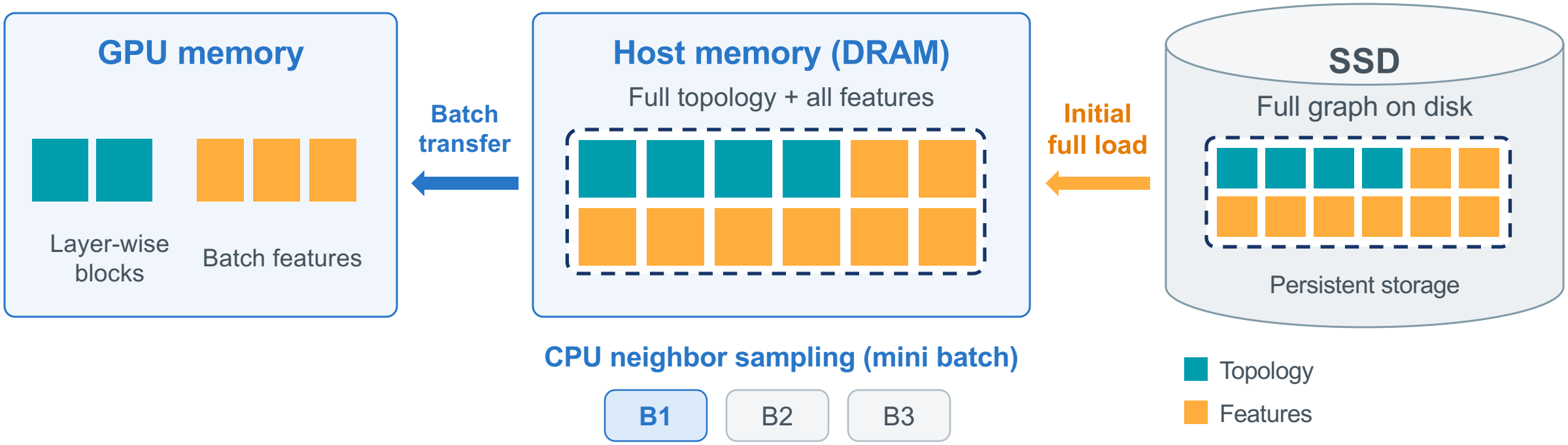
- Keep the full topology and features in host RAM
- Transfer only the current mini-batch



Sampled subgraph → GNN layer 1 → GNN layer 2 → Batch loss

DGL

- Keep the full topology and features in host RAM
- Transfer only the current mini-batch

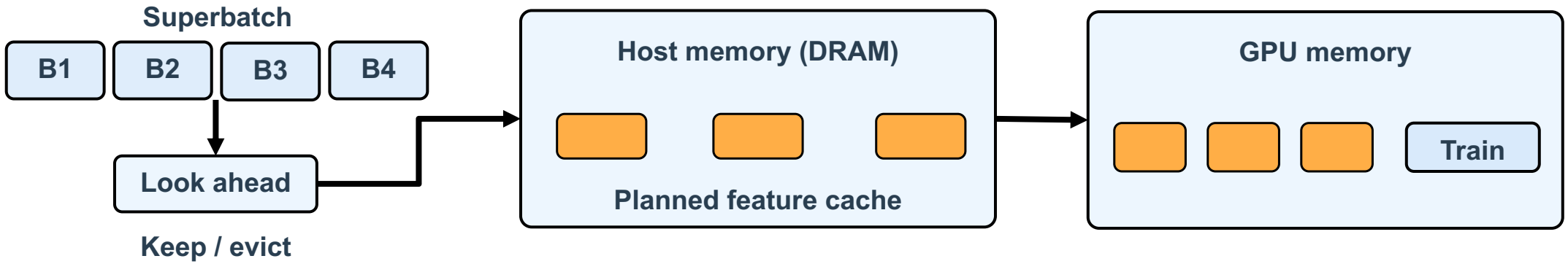


Block 1 + GNN layer 1 → Updated node features → Block 2 + GNN layer 2 → Batch loss

Reference: "M. Wang et al., "Deep Graph Library," arXiv:1909.01315, 2019."

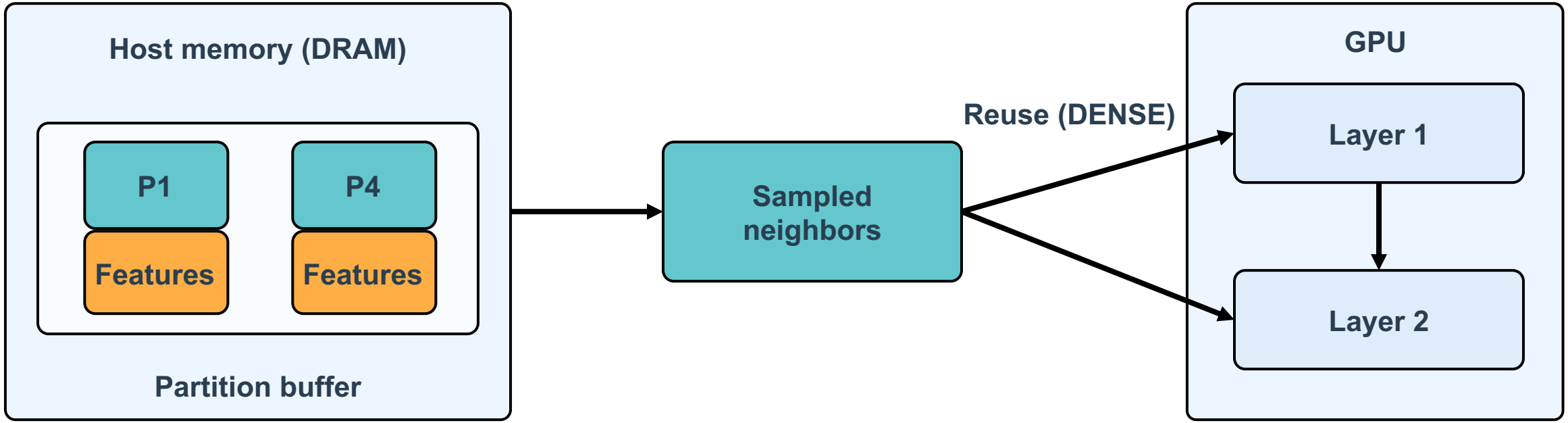
Ginex

- Sample a superbatch to identify future feature accesses
- Plan DRAM feature-cache replacement using Belady's policy
- Gather features and transfer each mini-batch to the GPU



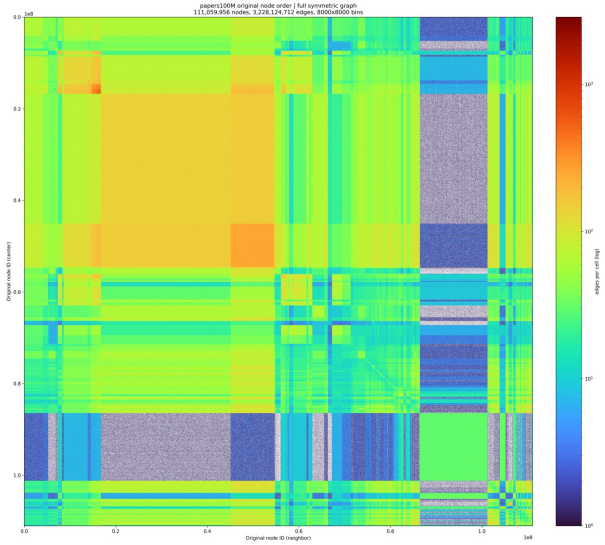
MariusGNN

- Load a subset of graph partitions into DRAM
- Reuse sampled neighbors across layers with DENSE
- Train on the GPU using mini-batches from the resident graph

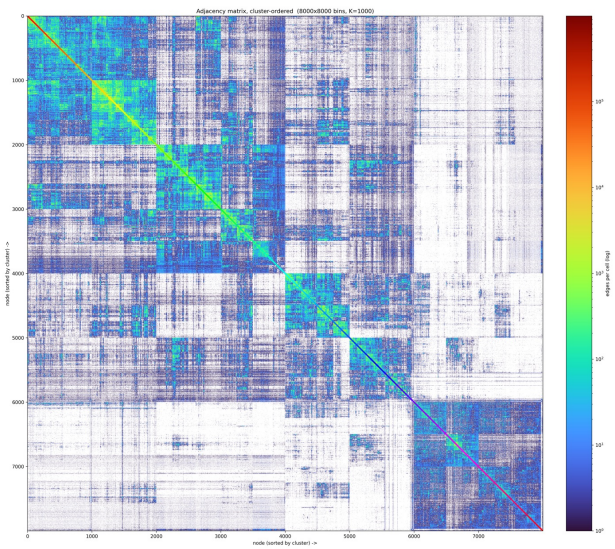


One Time Offline Graph Partitioning

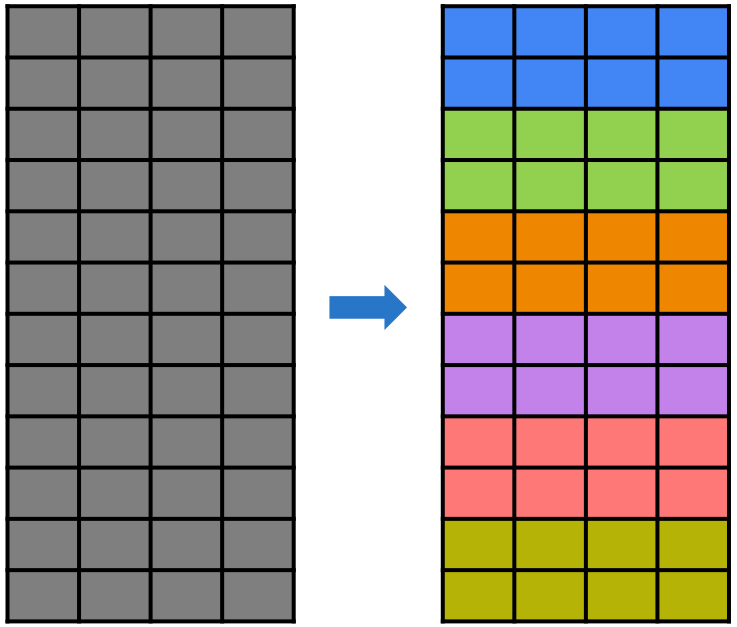
- Can use any other clustering method like Modularity
- One Time Offline Graph Partitioing with METIS
- Original node features are reordered according to the graph partitioning results



Original Data



Graph Partitioned Data

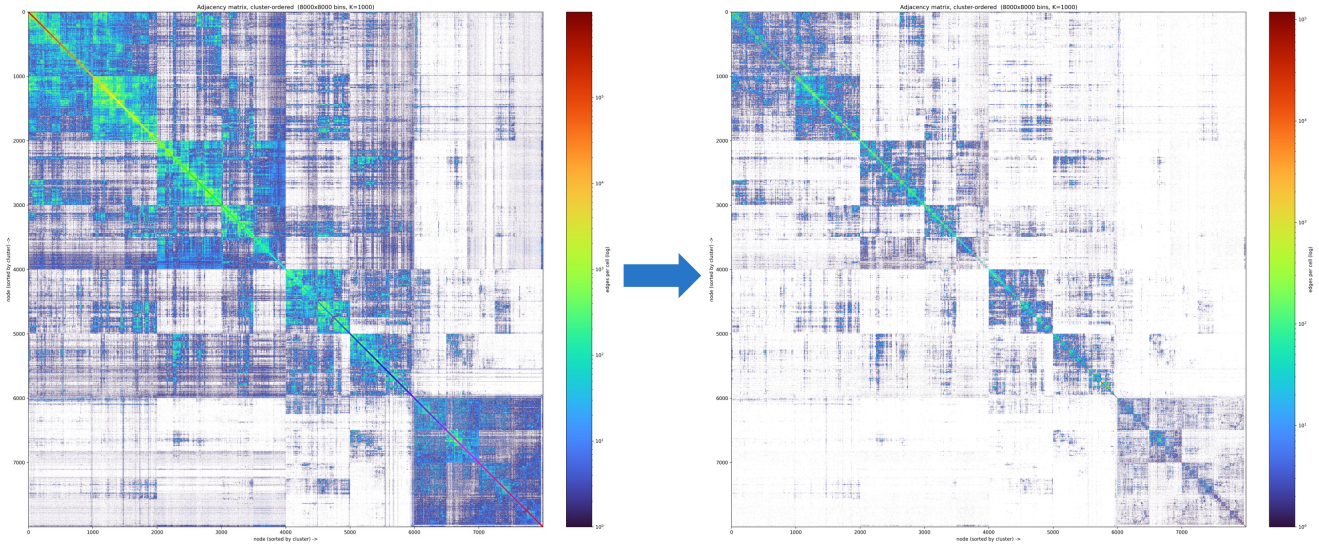


Original feature

Reordered feature

Structure Preserving Graph Sampling

- Use P_g to reduce non-training nodes while preserving all training nodes
- Sample within each community to preserve community structure
- Resample the graph at the beginning of each epoch

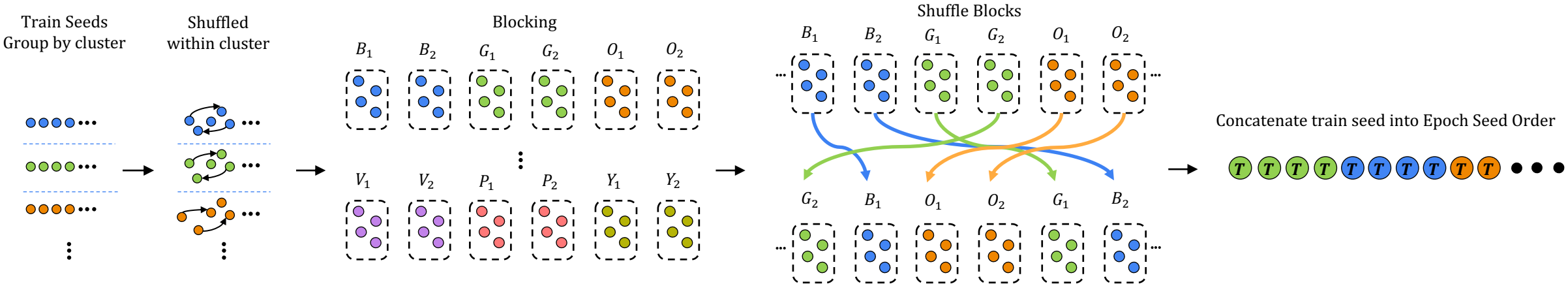


Before Graph Sampling Data

After Graph Sampling Data

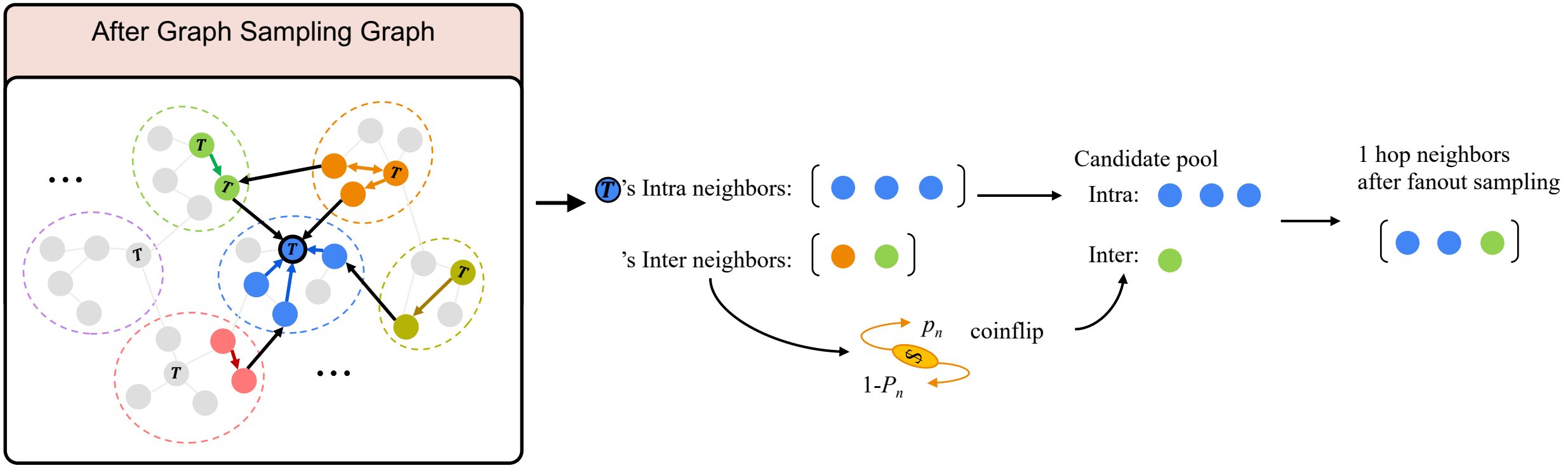
Make Blocks and Shuffle

- We make blocks which are consist of only train seeds
- Have 4 stages to make train seed order
  - Grouping by cluster
  - Shuffled within cluster
  - Blocking the train seeds with block size
  - Shuffled that blocks



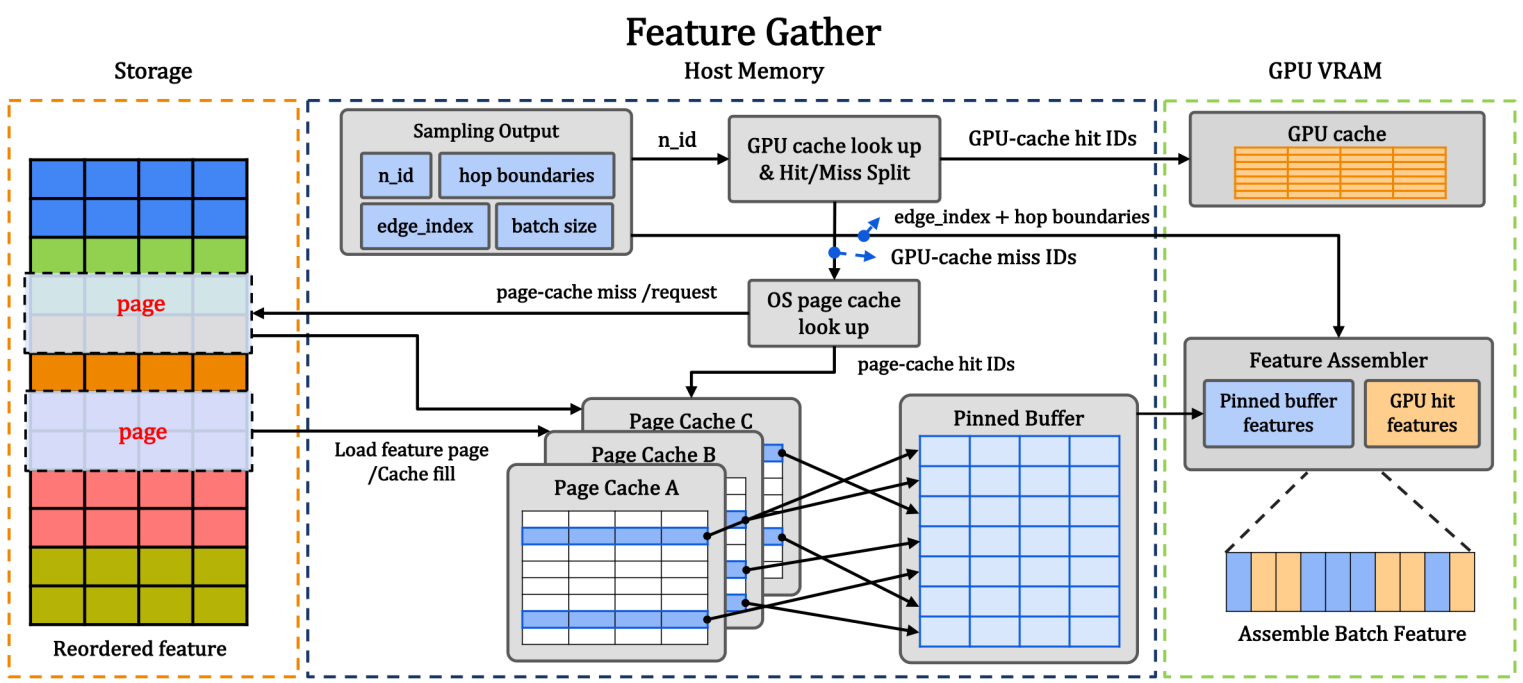
Community Aware Neighbor Sampling

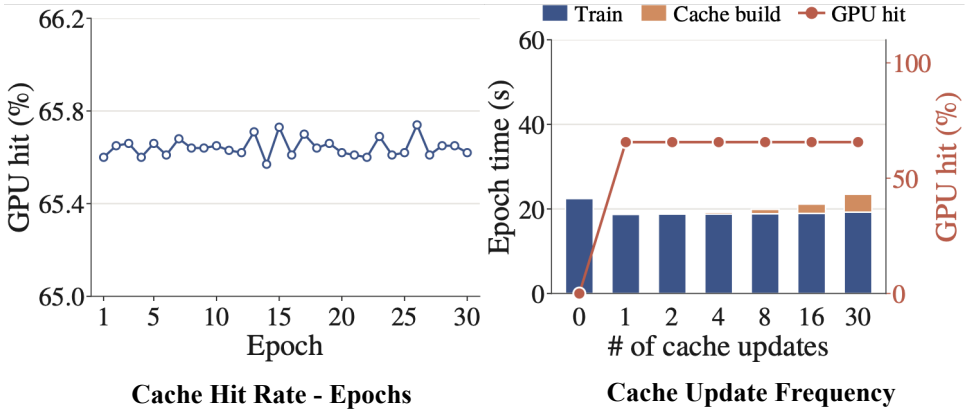
- Keep all intra-community neighbors and include inter-community neighbors with probability P_n in the candidate pool
- Uniformly sample the specified fanout from the candidate pool, repeating the process at each hop
- Reduce the neighbor search space and encourage shared neighbors to improve data locality



Out of Core Training

- Cache frequent node features using epoch-0 access counts and keep the GPU cache fixed
- Handle GPU cache misses via the OS page cache and SSD, gathering features into a pinned buffer
- Transfer gathered features to the GPU and combine them with cached features for GNN training



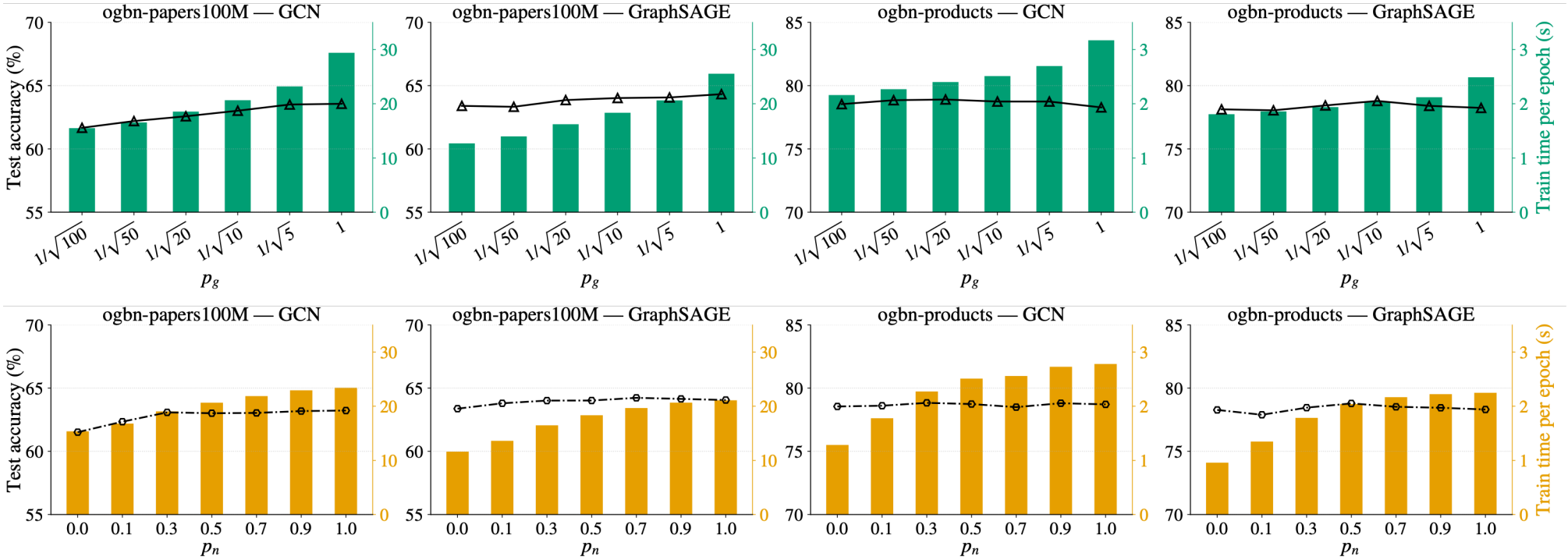


Cache Hit Rate - Epochs

Cache Update Frequency

Experiments

- **EQ 3 Sampling strategy**
  - Structure-preserving graph sampling reduces training time while comparable accuracy
  - Community-aware neighbor sampling lowers data preparation costs with comparable accuracy
  - The default (p_n = 0.5) balances training efficiency and predictive accuracy



Experiments

- **EQ 4 System optimization**
  - Larger blocks improve data locality and reduce training time, but lower mini-batch diversity can decrease accuracy
  - A default block size of 128 balances efficiency and accuracy, forming eight blocks per mini-batch of 1,024 target nodes
  - GPU caching reduces repeated feature transfers, with default cache sizes of 2 GB for papers and 100 MB for ogbn-products

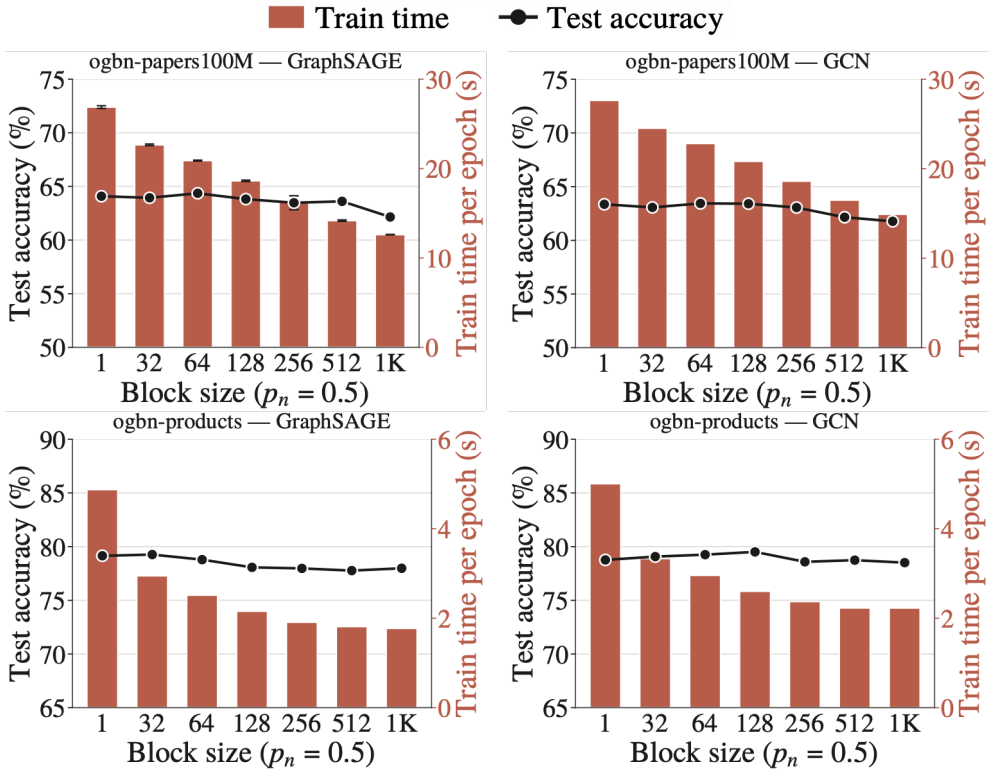


Figure 12 Effect of Block Size on Predictive Accuracy and Training Time

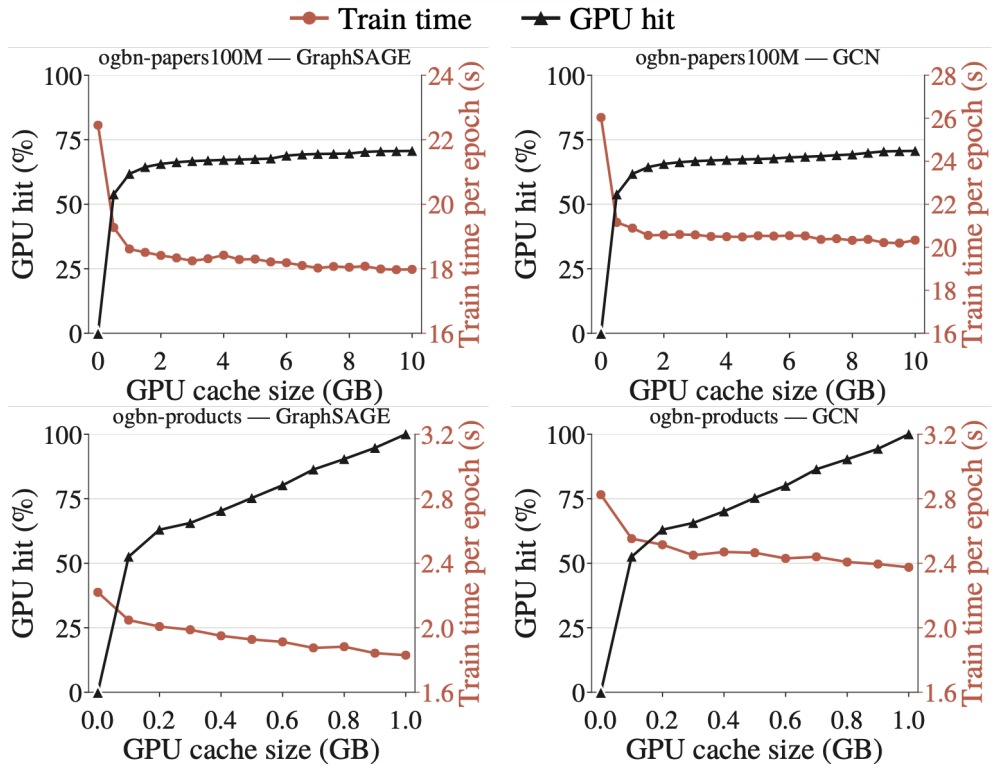


Figure 13 Effect of GPU Cache Size on Hit Rate and Training Time